

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Device Tree Metadata

How Bootloaders Pick DTBs and Apply Overlays



TOKYO, JAPAN / DEC. 11-13, 2025



Chen-Yu Tsai

- Works on ARM-based Chromebooks at Google
- Maintainer of wireless-regdb project
- Maintainer of Allwinner SoC within the kernel on the side

Development Kit Expansions

- Not covered in this session
- Identifying whether an expansion is attached and which one it is is the bigger problem
- Applying overlays based on identified expansion is solved

Open Problems

- Expansion overlays generic across baseboards
  - Please check out "Hotplug of Non-discoverable Hardware" by Bootlin from past ELC

Consumer Electronics Problem

- Many SKUs for a product due to component selection
  - 2x speaker codecs
  - 2x headphone codecs
  - 3x display panels
  - w/ or w/o keyboard backlight
- 2 * 2 * 3 * 2 = 24 SKUs
- Too many per-SKU DTB files
  - Duplicate content in DTBs
- Decompose into overlays by component
  - 1 base DTB
  - 8 overlays
  - Easy to do
- What about the SKU-specific model and compatible strings?
  - SKU-specific overlays?
  - Even more files?!?!

What Do We Need SKU-specific Strings For?

Compatible String

- Important for the bootloader
  - Identifies a set of DTBs and DTBOs to use
- Not so important for the kernel
  - A base device compatible string could be enough

Model String

- Not really used anywhere

Questions

- Can we decompose board DTS files into DTS files for the base design and overlays for component differences?
  - Likely yes
- Can we move from SoC DTS include files to SoC base DTBs?
  - Fails DT validation w/ missing top level compatible string
- Can we accept not-so-accurate top level model / compatibles in DTS files?
  - Bootloader may or may not insert the accurate values
- Can we store metadata (for the tools and bootloader) inside or close to the DTS files?

FIT Images



FIT Images

- Specifications now separate from U-boot
  - https://fitspec.osfw.foundation/
- Supported by
  - U-boot
  - coreboot
  - Android GBL (WIP)

- Bootloader provides compatible string
- Match against configuration nodes in FIT image
  - Configuration points to images to boot
    - Kernel
    - DTB + DTBOs
    - Ramdisk
    - etc.

FIT Image

```
images {
    kernel { ... };

    fdt-1 { /inc/"base.dtb" };

    fdt-2 { /inc/"p1.dtbo" };

    fdt-3 { /inc/"p2.dtbo" };
};
```

```
configurations {
    conf-1 {
        compatible = "dev-pan1";
        fdt = "fdt-1", "fdt-2";
    };

    conf-2 {
        compatible = "dev-pan2";
        fdt = "fdt-1", "fdt-3";
    };
};
```



Overlays in the Kernel Tree and make_fit.py

- **make_fit.py** builds a FIT image from a kernel image and DTBs
- **make_fit.py** can "decompose" composite DTBs into component DTB + DTBOs
- Compatible strings taken from composite DTBs

- What is there is no SKU-specific DTBO?
  - All configuration entries have the same compatible strings
  - Bootloader cannot tell them apart
- **Proposal**: put the SKU-specific compatible string mapping in a separate file
  - [RFC] arm64: dts: mediatek: Split Ciri into overlays
  - Actual format and tool changes TBD

Android



DTBs in Android

- DTB image in (vendor) boot image
- DTBO partition

DTB Image in (Vendor) Boot Image

- Boot image header gives how large the DTB image is and the offset into the boot image
- DTB image is just a bunch of DTBs concatenated together
- No lookup table; bootloader must read, parse, and match against each DTB
- Custom properties are commonly involved
- Matching logic is vendor specific
- Properties are vendor specific
- Dead end

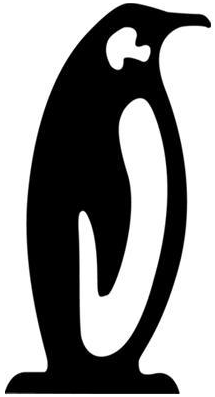
Ref: AOSP – DTB images

DTB and DTBO Partitions

- DT_TABLE format
- Each entry has metadata for matching
  - **NOT** a compatible string
  - Six u32 values
    - id
    - rev
    - 4x custom fields
- Bootloader can still inspect the DTB/DTBO if the metadata is insufficient

- Can we embed the six u32 values inside the DTB/DTBO?
  - board-id, board-rev, board-custom-1, ...
  - Tool can read the properties when building the image
- If not, can we store it *near* the source files?

Ref: AOSP - DTB and DTBO partitions



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025



Backup Slides



Device Tree Overlays



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Device Tree Overlays

What is possible:

- Small snippet of device tree
- Add device nodes
- Add or overwrite existing properties
- CANNOT delete device nodes or properties

Uses

- Describe expansion boards or attachments
  - Overlay can be applied by the bootloader or kernel on the fly
- Split out common parts of a family of DTBs
  - Reduce source file duplication
  - Reduce blob content duplication
  - Reduce DT validation time (?)

Device Tree Overlays - The Boundary

Overlay uses already in the kernel tree

- Expansion boards / HATs
- Carrier boards for System on Modules
- Onboard component differences
  - User selects correct composite DTB
  - Applied based on EEPROM identification
    - k3-am6xx-phycore-disable-eth-phy.dtso

Uses outside the kernel tree

- ChromeOS wants to model device SKUs component differences with overlays
- Some Android vendors model the SoC as the base and the board as an overlay
- Others?

Incomplete DTBs



Incomplete DTBs

Make the SoC.dtsi the base DTB

- Swappable SoC
  - Bootloader detects SoC and chooses correct base, then applies board overlay
- Avoid compiling the common SoC portions every time
- Minimize on-disk storage size
- Reduce DT validation times (?)

- No board compatible string
  - Fails DT validation
  - Firmware needs to insert correct string
  - Overlay needs to overwrite with correct string
  - Make tooling support "prepending" to a string array property

Proposal: Officially allow "incomplete" trees as a base by Doug Anderson